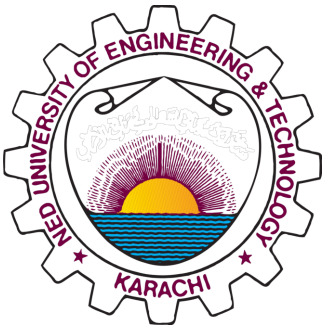


NED University of Engineering & Technology

Department of Software Engineering

Cloud Computing (SE-490)

Complex Engineering Activity (CEA)

Design, Deployment, and Cloud-Native Enhancement of a Three-Tier Web Application

Axon on Amazon Web Services

Technical Report

Submitted by:	Nabeed Ali	SE-23054
	Talha Hussain	SE-23062
Submitted to:	Miss Amna	
Course:	Cloud Computing (SE-490)	
Date:	November 2026	

Abstract

This report describes how we deployed Axon, a product-studio website, as a three-tier, cloud-native application on Amazon Web Services (AWS) in the Mumbai region (`ap-south-1`). The existing Vite front end was packaged with Nginx as the web tier, and we added a Node.js/Express application tier and an Amazon RDS PostgreSQL database tier. Both container tiers run in Docker on EC2 Auto Scaling groups spread over two Availability Zones in private subnets; a public Application Load Balancer is the only internet entry point, an internal load balancer fronts the application tier, and the managed database is private and encrypted. Every credential lives in AWS Secrets Manager. The entire environment is written as Terraform code, and a GitHub Actions pipeline authenticated by OpenID Connect (no stored AWS keys) tests, builds, publishes and rolls out each release automatically. We extended the deployment with CloudWatch logging and alarms, an S3 bucket and a cost analysis. A failure test showed the platform replacing a terminated instance with 56 of 60 requests served during the outage. To protect a limited credit budget the environment was destroyed after the evidence was captured and can be recreated with a single command.

Table of Contents

1 Introduction **1**

2 Choice of Cloud Provider **1**

3 System Architecture **1**

3.1 Overview 1

3.2 Network Design 1

3.3 Web and Application Tiers 1

3.4 Database Tier 2

4 Containerisation **2**

5 CI/CD Pipeline **2**

6 Security **3**

7 Load Balancing and Scalability **3**

8 Cloud-Native Enhancements **3**

8.1 Infrastructure as Code 3

8.2 Logging, Monitoring and Storage 3

8.3 Cost Estimation 3

9 Testing and Results **4**

10 Non-Functional Requirements **4**

11 Conclusion **5**

A Deployment Evidence **6**

A.1 Network and Subnets 6

A.2 Security Groups 6

A.3 Database, Secrets and IAM 7

A.4 Container Registry, Compute and Load Balancer 7

A.5 CI/CD Pipeline 8

A.6 Running Application 9

A.7 Object Storage 9

B Source Files **9**

1 Introduction

Axon is a product-studio website: a single-page application built with Vite. For this activity we turned it into a genuine three-tier system. The static site is served by an Nginx web tier; a new Node.js API provides a health endpoint, newsletter subscription and a contact form; and the data is stored in a managed PostgreSQL database (Table 1). The goal was to run the system the way a production team would: isolated networking, containers built from version control, no secrets in code, automatic deployments, health-based self-healing and monitoring, all reproducible from code.

Table 1: *Application components and their tiers*

Tier	Component	Technology
Web	Axon site (`web/`, `src/`)	Vite build served by Nginx in a Docker container
Application	REST API (`api/`)	Node.js 20, Express, `pg`, AWS SDK for S3
Database	Application data	Amazon RDS for PostgreSQL 16, private and encrypted

2 Choice of Cloud Provider

We selected AWS and deployed in `ap-south-1` (Mumbai). Four reasons decided it. First, AWS offers a managed service for every requirement of the activity (VPC, load balancing, Auto Scaling, RDS, ECR, Secrets Manager, CloudWatch, S3), so very little undifferentiated infrastructure had to be hand-built. Second, Terraform's AWS provider is mature, which makes the deployment reproducible and destroyable, an essential property under our limited budget of about US$38 of free-plan credit. Third, Mumbai is the closest AWS region to Pakistan and therefore gives the lowest latency to users, even though its prices are marginally above us-east-1 (for example t3.micro at US$0.0112 per hour). Fourth, GitHub Actions can assume an AWS role through OpenID Connect (OIDC), which let us build a pipeline that stores no long-lived access keys.

3 System Architecture

3.1 Overview

Figure 1 shows the complete deployment. A request enters through the Internet Gateway and reaches the public Application Load Balancer (ALB). The ALB forwards it to an Nginx container in the web tier, which serves the static site and reverse-proxies `/api/` calls to an *internal* ALB in front of the application tier. The Node.js API reads and writes data in RDS PostgreSQL. Because each hop only accepts traffic from the hop before it, the application tier is reachable solely from the web tier, and the database solely from the application tier.

3.2 Network Design

We created a dedicated VPC (`10.0.0.0/16`) with eight subnets over two Availability Zones (Table 2). The public route table sends internet traffic to the Internet Gateway. The web and application subnets send outbound traffic through one shared NAT gateway, which lets instances pull container images and reach AWS APIs without being reachable from the internet. The database route table has *no* default route at all. A free S3 gateway endpoint keeps image-layer downloads on the AWS network and off the billed NAT gateway.

Table 2: *Subnet layout of the Axon VPC*

Subnet tier	CIDR (1a / 1b)	Default route	Resources
Public	10.0.1.0/24, 10.0.2.0/24	Internet Gateway	Public ALB, NAT gateway
Private web	10.0.11.0/24, 10.0.12.0/24	NAT gateway	Web Auto Scaling group
Private app	10.0.21.0/24, 10.0.22.0/24	NAT gateway	Internal ALB, app Auto Scaling group
Private database	10.0.31.0/24, 10.0.32.0/24	none	RDS subnet group

3.3 Web and Application Tiers

Each compute tier is an Auto Scaling group (ASG) of `t3.micro` instances (minimum 2, desired 2, maximum 3) created from a launch template that requires IMDSv2 and assigns no public IP. At boot a short script installs Docker, logs in to ECR and runs the tier's container; the application tier first fetches its database credential from Secrets Manager.

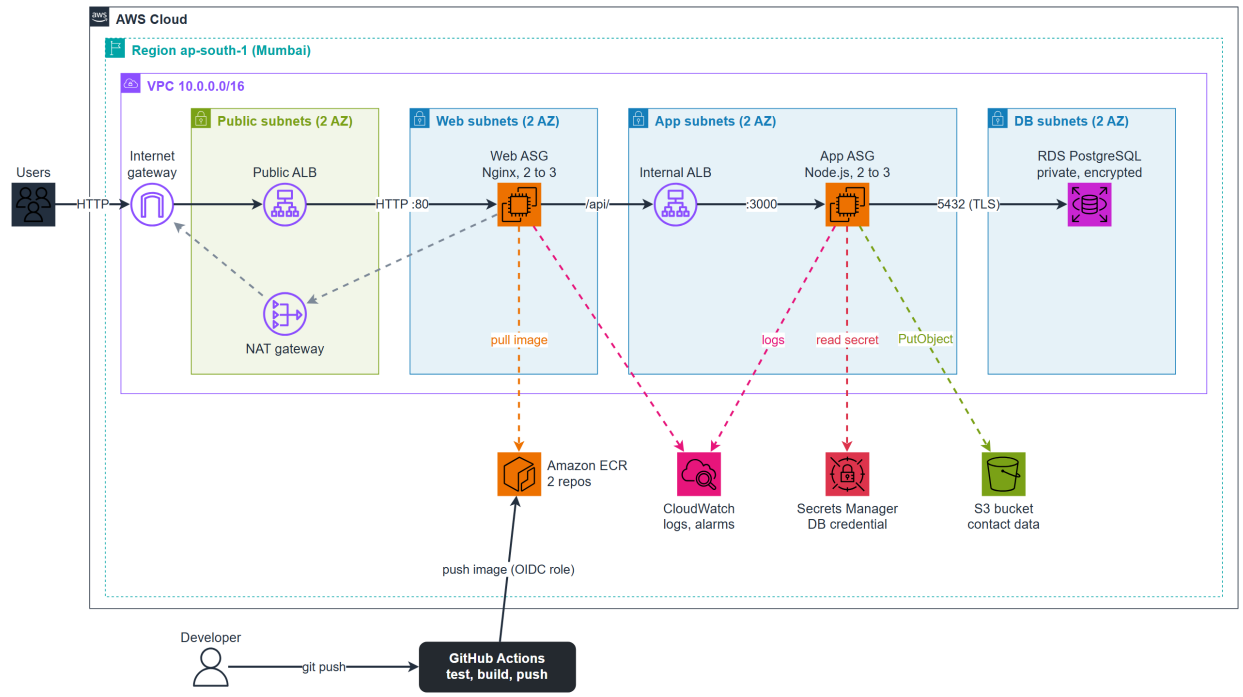


Figure 1: *Axon deployment architecture on AWS: VPC across two Availability Zones, public and internal load balancers, web and application Auto Scaling groups, private database, managed services and the GitHub Actions deployment path.*

Instances have no SSH access; administration uses AWS Systems Manager Session Manager. The web ASG registers with the public ALB, and the app ASG with the internal ALB, whose target groups use the container's `/healthz` endpoint (liveness without a database dependency).

3.4 Database Tier

The data tier is an Amazon RDS for PostgreSQL 16 instance (`db.t3.micro`) in the two database subnets. It is not publicly accessible, storage is encrypted, and RDS itself generates the master password and stores it in Secrets Manager. The API creates its schema on start-up and exposes `/api/health`, which performs a real database check and is used as an evidence endpoint.

4 Containerisation

Every tier is a Docker image. The web image is a two-stage build: the first stage compiles the site with Vite, and the second copies only the compiled files into `nginx:1.27-alpine`. Nginx renders its configuration from a template with `envsubst`, so the same image works locally and on AWS; it adds security headers, enables compression and long-lived caching of hashed assets, proxies `/api/` and answers the load balancer's `/healthz` check. The API image is also multi-stage, installs production dependencies only, runs as the unprivileged `node` user and declares a `HEALTHCHECK`. A `docker-compose.yml` starts web, API and PostgreSQL locally on two separate networks, so the web container cannot reach the database, with a single command.

5 CI/CD Pipeline

The workflow `.github/workflows/deploy.yml` runs on every push to `main` (Figure 2). The *test* job runs the 90 site tests, the production build and the 10 API tests. The *deploy* job then assumes an AWS role through OIDC, logs in to ECR, builds both images and pushes each with two tags, the Git commit SHA and `latest`. It starts a rolling instance refresh on both ASGs (50% minimum healthy, 120 s warm-up), waits for it to finish, looks up the public ALB's DNS name and smoke-tests it. A concurrency group prevents two deployments from overlapping, and because instances pull `latest` at boot, scale-out events also receive the newest release. The role's trust policy accepts only the `main` branch of this repository, pinned by its immutable repository ID, and the pipeline needs no AWS secret in GitHub at all.

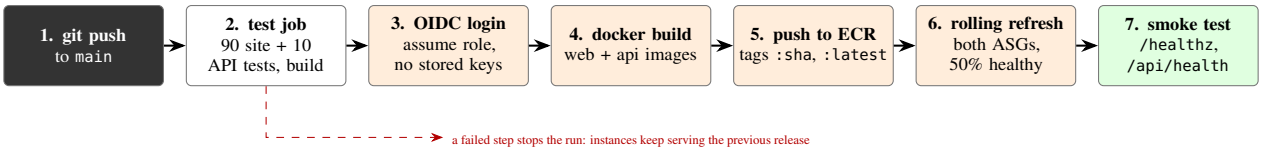


Figure 2: *Stages of the GitHub Actions pipeline.*

6 Security

Security is applied in layers. At the network level only the public ALB and the NAT gateway sit in public subnets; instances and the database have no public IP address, and the database subnets have no internet route. At the firewall level one security group per tier forms a chain in which each tier accepts traffic only from the tier in front of it (Table 3); because the rules reference security groups instead of IP addresses they stay correct as instances are replaced. No security group allows SSH.

Table 3: *Security group rules*

Security group	Inbound rule	Attached to
`axon-alb-sg`	TCP 80 from the internet	Public ALB
`axon-web-sg`	TCP 80 from the public ALB group	Web instances
`axon-internal-alb-sg`	TCP 80 from the web group	Internal ALB
`axon-app-sg`	TCP 3000 from the internal ALB group	App instances
`axon-db-sg`	TCP 5432 from the app group	RDS instance

Secrets are handled by AWS Secrets Manager. RDS generates the master password and keeps it in a managed secret; the application's instance role may read *only that secret*, and the boot script writes it to a root-only file used by the container, so nothing sensitive appears in Git, images, user-data or CI. IAM follows least privilege: separate web and app instance roles (ECR pull for their own repository, writes to their own log group, and for the app `s3:PutObject` on one prefix) and a CI role that can only push the two images and refresh the two ASGs. Data is encrypted at rest in EBS, RDS and S3, S3 public access is blocked, IMDSv2 is required, ECR scans every pushed image, and the containers run as non-root users.

7 Load Balancing and Scalability

Two ALBs distribute traffic over healthy instances in both Availability Zones and stop routing to any target that fails its health check. Because the ASGs use ELB health checks, an instance that fails is terminated and replaced automatically. Each ASG keeps between two and three instances and uses a target-tracking policy that holds average CPU utilisation at 50%; two instances per tier mean one runs in each zone at all times. Six consecutive calls to `/api/health` through the public ALB alternated between two application instances in `ap-south-1a` and `ap-south-1b`, confirming that load is spread across zones.

8 Cloud-Native Enhancements

8.1 Infrastructure as Code

The complete environment (86 resources) is defined in Terraform: network, security groups, ALBs, ASGs, RDS, ECR, S3, IAM, the GitHub OIDC role, CloudWatch and budget alerts. `scripts/up.ps1` creates and deploys everything with one command and `scripts/down.ps1` destroys it, which we used to stop all charges after the evidence was captured.

8.2 Logging, Monitoring and Storage

Container logs go to CloudWatch Logs through the Docker `awslogs` driver with seven-day retention. A CloudWatch dashboard and four alarms (ALB 5xx errors, unhealthy hosts in the web and app target groups, and RDS CPU) notify an SNS topic by e-mail, and AWS Budgets warns at US$15 and US$30. An encrypted, private, versioned S3 bucket stores every contact-form submission written by the API.

8.3 Cost Estimation

Table 4 estimates the monthly cost of the full environment at 730 hours using AWS Price List rates for `ap-south-1`; the total is about US$148 per month, or US$4.9 per day.

Table 4: *Estimated monthly cost (US$, ap-south-1)*

Service	USD/month	Service	USD/month
4 × EC2 t3.micro	32.70	Public IPv4 addresses (3)	10.95
NAT gateway (+10 GB data)	41.44	EBS, 32 GB gp3 (approx.)	2.92
2 × Application Load Balancer	36.29	S3, CloudWatch, Secrets, ECR (approx.)	1.90
RDS db.t3.micro + 20 GB (approx.)	21.60	**Total**	**147.80**

Two optimisations are implemented: a single shared NAT gateway instead of one per zone saves US$40.9 per month (28%), and the S3 gateway endpoint keeps image traffic off the metered NAT; and the environment is ephemeral, so a few days of testing cost roughly US$5 per day instead of US$148 per month. Proposed next: merge the internal ALB into the public one with path-based routing (about US$18 per month) and move to Graviton `t4g.micro` instances (about 20% cheaper).

9 Testing and Results

Testing covered four levels (Table 5); the console and application screenshots are in Appendix A.

Table 5: *Test results*

Test	Result
Unit and integration	API: 10/10 pass (validation, idempotent subscribe, database-failure handling); site: 90/90 pass.
Local three-tier run	`docker compose up`: site 200, `/api/health` reports the database up, a duplicate subscription is handled, and the row is visible in PostgreSQL.
Infrastructure	`terraform validate` passed; apply succeeded; the follow-up plan showed no drift.
Automated deployment	Run 1 failed at the OIDC step (CloudTrail showed GitHub now issues subject claims with immutable IDs; the trust policy was updated). Run 2: all jobs green in 11 min 44 s. The live site footer then reported the new commit.
Write path	POST `/api/subscribe` through ALB → Nginx → internal ALB → API → RDS returned 201.
Failure injection	One app instance was terminated while a request was sent every 2 s for 120 s: 56 of 60 requests succeeded; the 4 failures (HTTP 502 or time-out) all occurred in the first 16 s, before the ALB health check removed the dead target, and the ASG had a healthy replacement at the end.

10 Non-Functional Requirements

Table 6 maps design decisions to each quality attribute, together with the known limitation and its improvement path.

Table 6: *How the solution addresses the non-functional requirements*

Requirement	Design and implementation decisions	Known limitation → improvement
Scalability	Stateless containers (state only in RDS and S3); ALB spreads load; ASGs scale 2→3 on CPU target tracking; new instances boot from `latest`.	Scale-out was configured but not load-tested; single database. → read replicas, RDS Proxy, request-based scaling.
Availability and reliability	Two AZs per tier, two ALBs, at least two instances per tier, ELB health checks with automatic replacement (measured in the failure test).	RDS is single-AZ and one NAT is an egress single point of failure. → Multi-AZ RDS, a NAT per zone.
Elasticity	Target-tracking policies add and remove instances; the whole stack is created and destroyed by Terraform.	Small maximum size, CPU-only trigger. → larger bounds, scheduled scaling.
Security	Section 6: per-tier security groups, private subnets, least-privilege IAM, managed secret, OIDC pipeline, encryption, no SSH.	Traffic is HTTP (no domain or certificate); app uses the DB master user. → HTTPS with ACM and WAF; dedicated DB role.
Maintainability and deployability	Everything is in Git; one push tests, builds, pushes and rolls out with no manual step; commit-SHA tags allow rollback.	Rollback is manual (re-tag). → automatic rollback on a failed smoke test, blue/green.
Performance and latency	Mumbai region; Nginx compression and long-lived caching; ALBs in the same region; S3 endpoint avoids a NAT detour.	No CDN or cache; burstable t3.micro CPU. → CloudFront, ElastiCache, larger instances.
Cost-efficiency	One shared NAT, t3.micro and db.t3.micro, gp3 volumes, 7-day log retention, ECR lifecycle rule, S3 endpoint, budget alerts, and teardown after evidence capture.	Idle cost is dominated by NAT and ALBs. → consolidate ALBs, Savings Plans, Graviton.
Portability	Docker images hold all dependencies; configuration by environment variables; the same images run under compose, on EC2, and could run on ECS or EKS.	Terraform, ALB, ASG, RDS and OIDC trust are AWS-specific. → add Kubernetes manifests.

11 Conclusion

Axon now runs on AWS as a secure, self-healing three-tier application. The network is segmented into public, web, application and database subnets across two Availability Zones; the web and application tiers run as containers in Auto Scaling groups behind two load balancers; the data lives in a private, managed PostgreSQL database; and every push to GitHub is tested and deployed automatically without stored credentials. All mandatory requirements and four enhancements (Terraform, CloudWatch, S3 and cost analysis) were delivered. Honest gaps remain: no HTTPS, a single-AZ database and NAT gateway, CPU-only scaling that was not load-tested, local Terraform state, and no Kubernetes; each has an improvement path in Table 6.

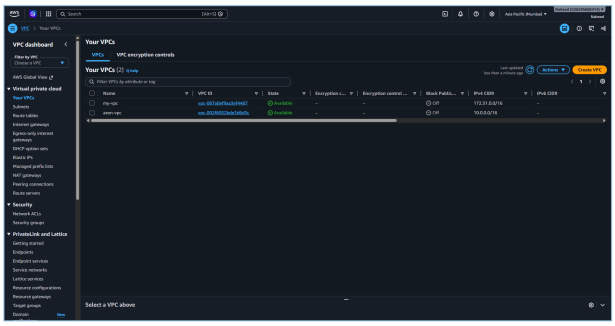
References

[1] Amazon Web Services, *Elastic Load Balancing User Guide*. https://docs.aws.amazon.com/elasticloadbalancing/
[2] Amazon Web Services, *Amazon EC2 Auto Scaling User Guide*. https://docs.aws.amazon.com/autoscaling/ec2/
[3] Amazon Web Services, *Amazon RDS User Guide*. https://docs.aws.amazon.com/rds/
[4] HashiCorp, *Terraform AWS Provider*. https://registry.terraform.io/providers/hashicorp/aws/
[5] GitHub, *Configuring OpenID Connect in Amazon Web Services*. https://docs.github.com/actions

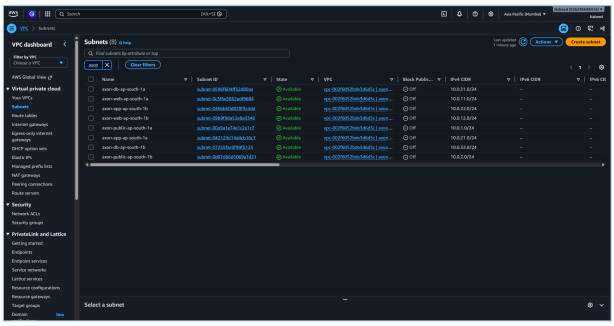
A Deployment Evidence

The figures below were captured from the AWS console, GitHub and the running application while the stack was deployed.

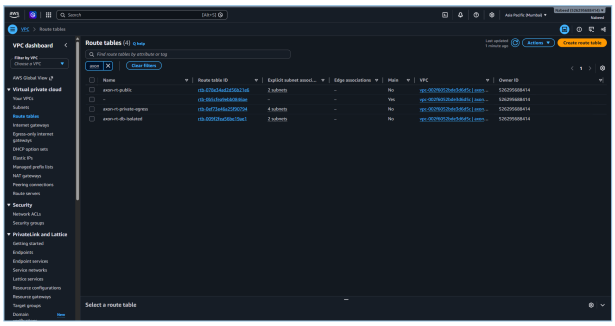
A.1 Network and Subnets



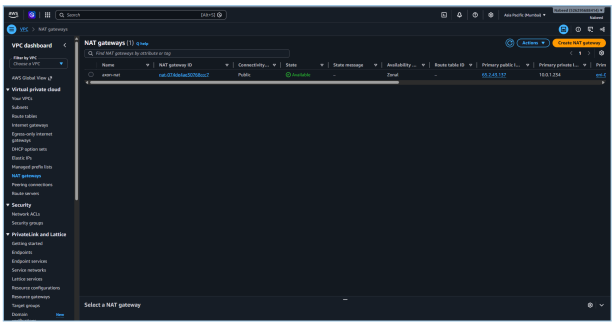
VPC `axon-vpc` (10.0.0.0/16)



Eight subnets over two Availability Zones

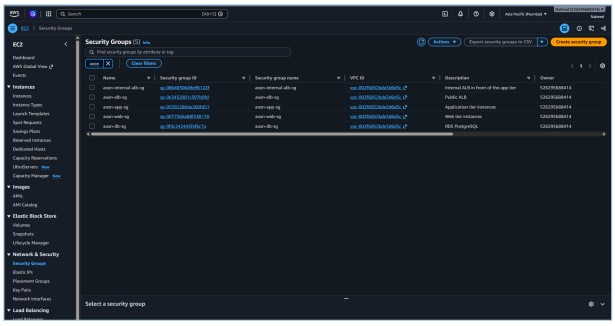


Route tables: public, private egress and isolated database

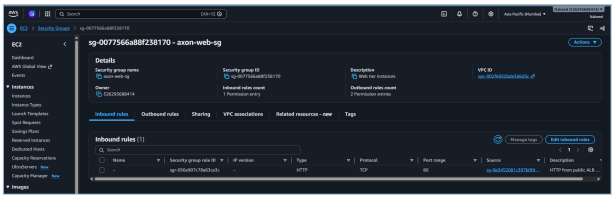


NAT gateway in a public subnet

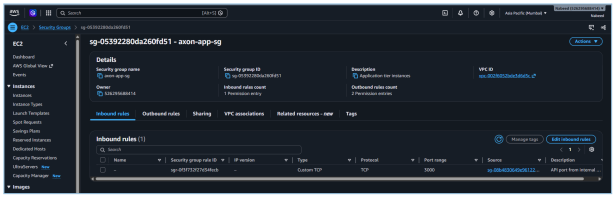
A.2 Security Groups



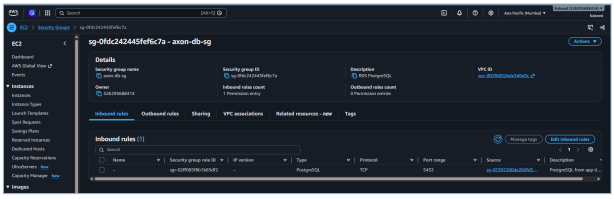
One security group per tier



Web tier: inbound only from the public ALB group

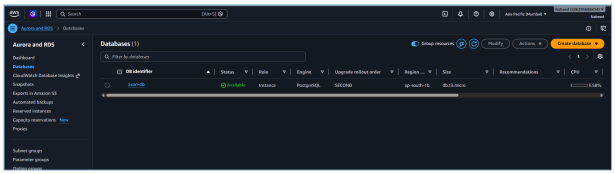


App tier: port 3000 only from the internal ALB group

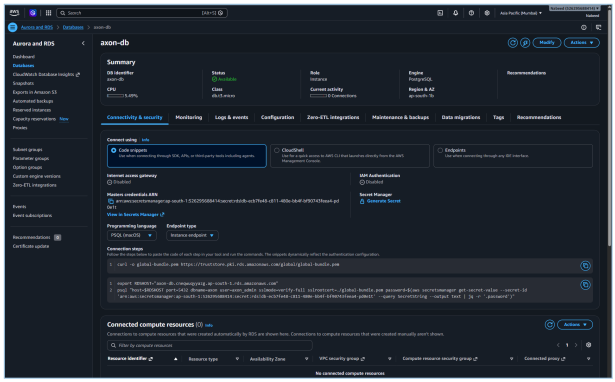


Database: port 5432 only from the app group

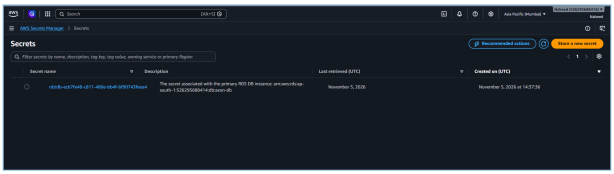
A.3 Database, Secrets and IAM



RDS PostgreSQL instance `axon-db`

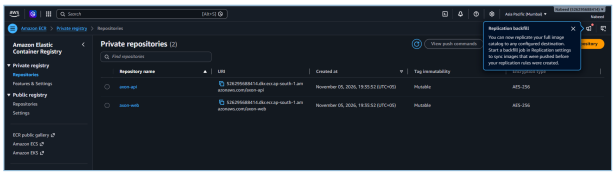


RDS detail: private, not publicly accessible

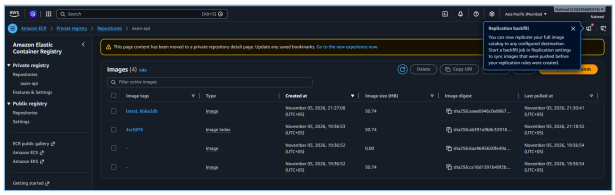


Secrets Manager holds the RDS-managed credential

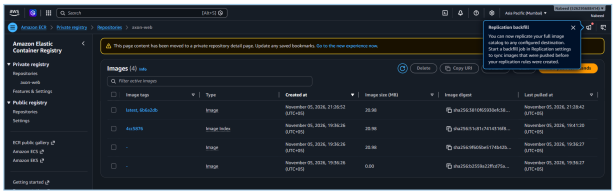
A.4 Container Registry, Compute and Load Balancer



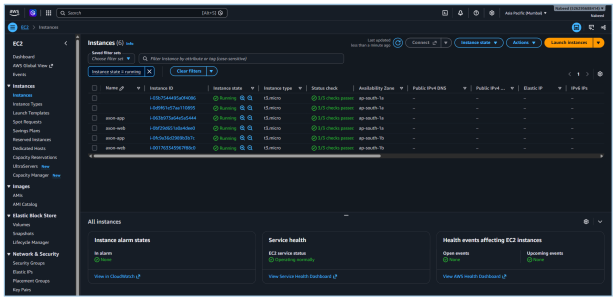
Amazon ECR repositories



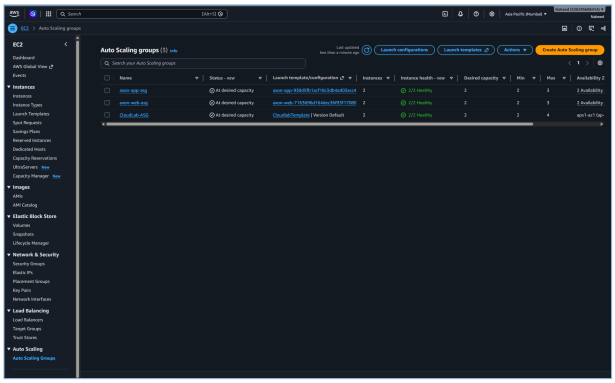
API images tagged with commit SHA and latest



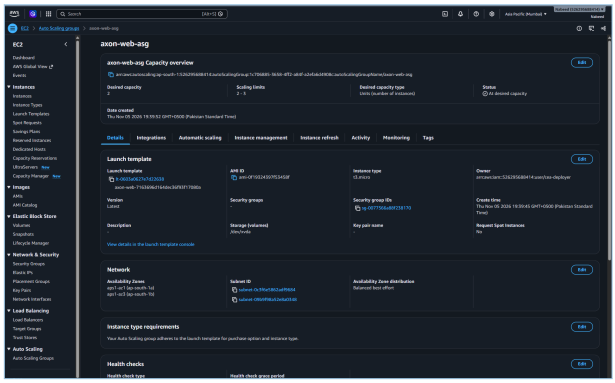
Web images tagged with commit SHA and latest



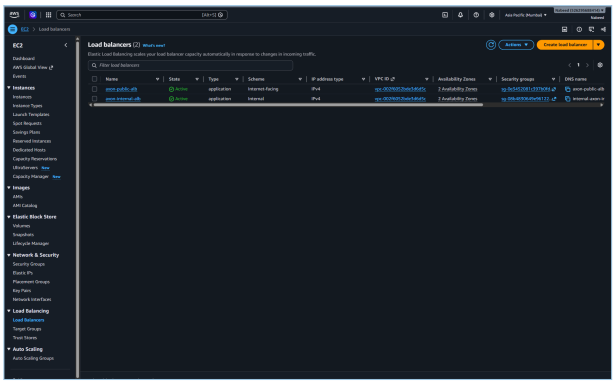
Running web-tier and app-tier instances



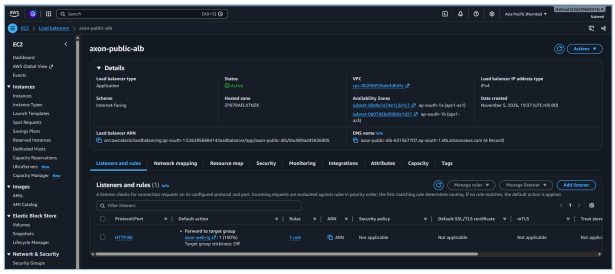
Auto Scaling groups for both tiers



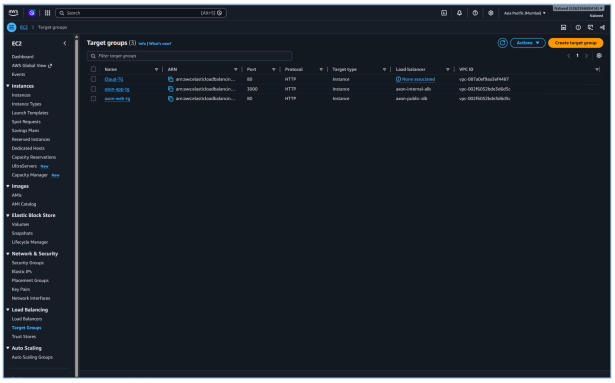
Web ASG: capacity, launch template, target group



Public and internal Application Load Balancers

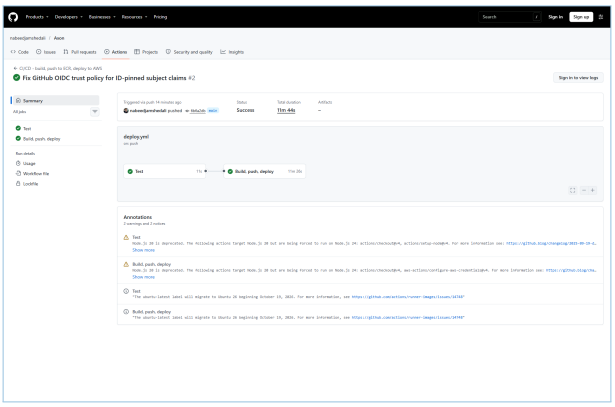


Public ALB: listener, subnets, security group



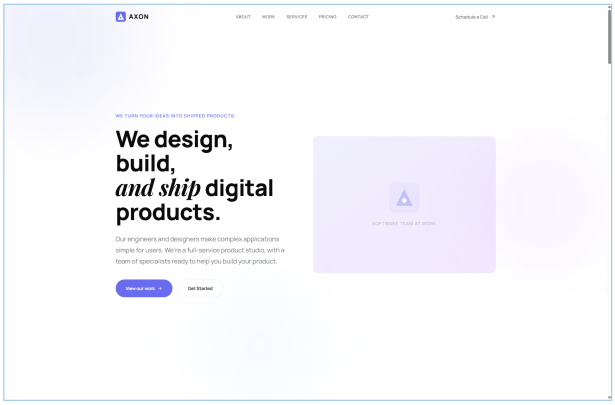
Target groups with healthy targets

A.5 CI/CD Pipeline

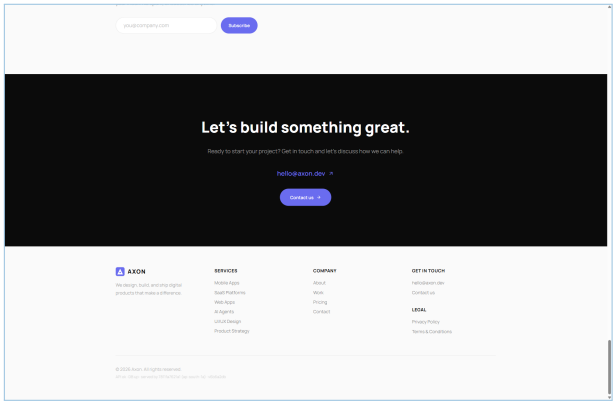


Successful GitHub Actions run: test, build, push, deploy

A.6 Running Application

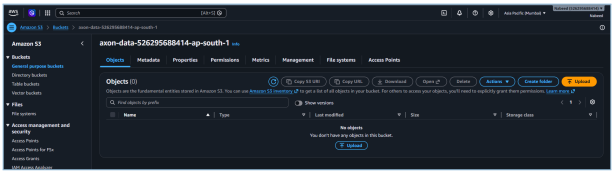


Application served through the public ALB



Footer status: API ok, DB up, serving instance, AZ and commit

A.7 Object Storage



S3 bucket for contact submissions (private, encrypted)

B Source Files

File	Purpose
`web/Dockerfile`, `api/Dockerfile`	Image definitions for the web and application tiers
`web/default.conf.template`	Nginx configuration and reverse proxy
`docker-compose.yml`	Local three-tier setup on isolated networks
`.github/workflows/deploy.yml`	CI/CD pipeline
`infra/terraform/*.tf`	All AWS infrastructure as code
`scripts/up.ps1`, `scripts/down.ps1`	One-command create and destroy of the stack

Source code repository: https://github.com/nabeedjamshedali/Axon